PdfCraft for Android — Porting Specification

Oct 14, 2026 · @Tendai

Purpose and how to use this document

This specification tasks an AI coding agent with building a native Android app from the `tendai98/pdfcraft` repository, reusing the existing Rust engine and egui shell unchanged wherever possible. The executor is an autonomous agent (Claude Code or equivalent) with a shell, the Rust toolchain, the Android NDK and SDK, and at least one Android device or emulator. A human owner reviews each phase and answers the reserved decisions listed at the end.

Definition of done (v1): a signed, installable APK/AAB for `arm64-v8a` that opens a PDF chosen through the Android document picker, renders and scrolls it with touch, supports the read-and-organize feature set below, and saves the result back to a user-chosen location. The build is reproducible from a fresh clone in CI. Everything desktop-only is either gated out cleanly or stubbed with a visible "not available on Android" state, never a crash.

How to execute: work through the phases in order. Each phase ends with its acceptance criteria met and committed. Do not start a phase whose dependencies are unmet. When a task says "ask the owner", stop and ask rather than guess. Prefer changes that could be upstreamed to `storytold/pdfcraft` over Android-only forks of shared code.

Source codebase

The source is github.com/tendai98/pdfcraft, a fork of storytold/pdfcraft, at commit `c1a41cc` (2026-10-14), workspace version 0.7.1. It is a Cargo workspace, Rust edition 2024, `rust-version = "1.95"`, resolver 3, with `unsafe_code = "forbid"` across the workspace. All PDF engine code is pure Rust: the renderer is `hayro`, cryptography is RustCrypto, JavaScript is `boa`, OCR is `ocrs` / `rten`. There are no C dependencies in the engine and no `-sys` crates outside the desktop shell. The workspace already builds for `wasm32` (`apps/pdfcraft-web`), which is a stricter target than Android.

Members and their Android status (line counts are `.rs` lines at `c1a41cc`):

Crate	Lines	What it is	Android status
`crates/ui-egui`	78,348	The egui shell: all screens, panels, dialogs, file flows	Reuse; needs a platform seam for files, clipboard, fonts, storage
`crates/engine`	18,186	Façade: session, documents, tool catalogue	Reuse unchanged
`crates/automation`	15,194	Headless tool set and MCP server for agents	Exclude from the Android build
`crates/render`	12,534	Page rendering (hayro), raster work queue	Reuse unchanged; measure speed on device
`crates/edit`, `sign`, `xfa`, `forms`, `organize`, `js`, `cos`, `annot`, `fonts`, `a11y`, `redact`	3,680–10,391 each	Engine features	Reuse unchanged
`crates/optimize`, `xfdf`, `create`, `export`, `filters`, `measure`, `preflight`, `crypt`, `audit`, `content`, `compare`, `ocr`, `geom`, `model`	215–2,084 each	Engine features	Reuse unchanged
`crates/print`	1,552	Print layout and system spooler	Layout reuse; spooler stubbed (desktop-only: CUPS, PowerShell)
`crates/platform`	686	OS services: desktop theme, staging	Add Android branch or gate out
`apps/pdfcraft`	3,081	Desktop entry point	Not reused; reference only
`apps/pdfcraft-cli`	2,714	Command-line and agent driver	Exclude
`apps/pdfcraft-web`	251	Browser shell around `PdfCraftApp`	The template for the Android shell

Crate	Lines	What it is	Android status
`vendor/`	—	Patched `winit` 0.30.13, `egui-wgpu` 0.36.2, `hayro*`, `lopdf`	Keep as is; patches are desktop-specific and inert on Android

Key facts the executor will rely on:

- UI stack: `egui` / `eframe` 0.36, drawing through `wgpu` with `glow` as fallback. Both backends support Android. The vendored `winit` already declares the `android-native-activity` feature.
- File dialogs: `rfd` 0.17 `AsyncFileDialog`, 45 call sites across 10 files in `crates/ui-egui/src` (`files.rs`, `combine_ui.rs`, `create_ui.rs`, `ocr_ui.rs`, `print_ui.rs`, `actions_ui.rs`, `content_ui.rs`, `marks_ui.rs`, `measure_ui.rs`, `comment_props.rs`). `rfd` has no Android backend.
- Files already travel as `(String, Vec<u8>)` name-and-bytes pairs (`FileRequest`, `use_files`, `startup_inbox`), which is exactly what Android content URIs resolve to.
- Platform gating today is two-way: `cfg(target_arch = "wasm32")` (270 sites in `ui-egui`) versus native. Android falls on the native side of that split and so inherits desktop assumptions (paths, `std::fs`, `arboard`, `rfd`).
- Native-only dependencies of `ui-egui`: `arboard` (clipboard, no Android support), `pollster`, `skrifa`, `getrandom`. `system_fonts.rs` hardcodes macOS, Linux and Windows font paths for script fallback.
- Rendering runs on a thread pool (`rayon`; `crates/render/src/raster/work_queue.rs`), so multi-core phones are used.
- OCR is on-device (`rten` model); model size and inference time on a phone are unmeasured.
- Licence: MIT OR Apache-2.0. The ArtCraft name and logos in `docs/brand/` are trademarks; forks must remove them (README, `docs/brand/LICENSE-brand.txt`).

Target

The target is a native Android app, tablet-first, phone-capable, built from the same Rust workspace as the desktop and web builds.

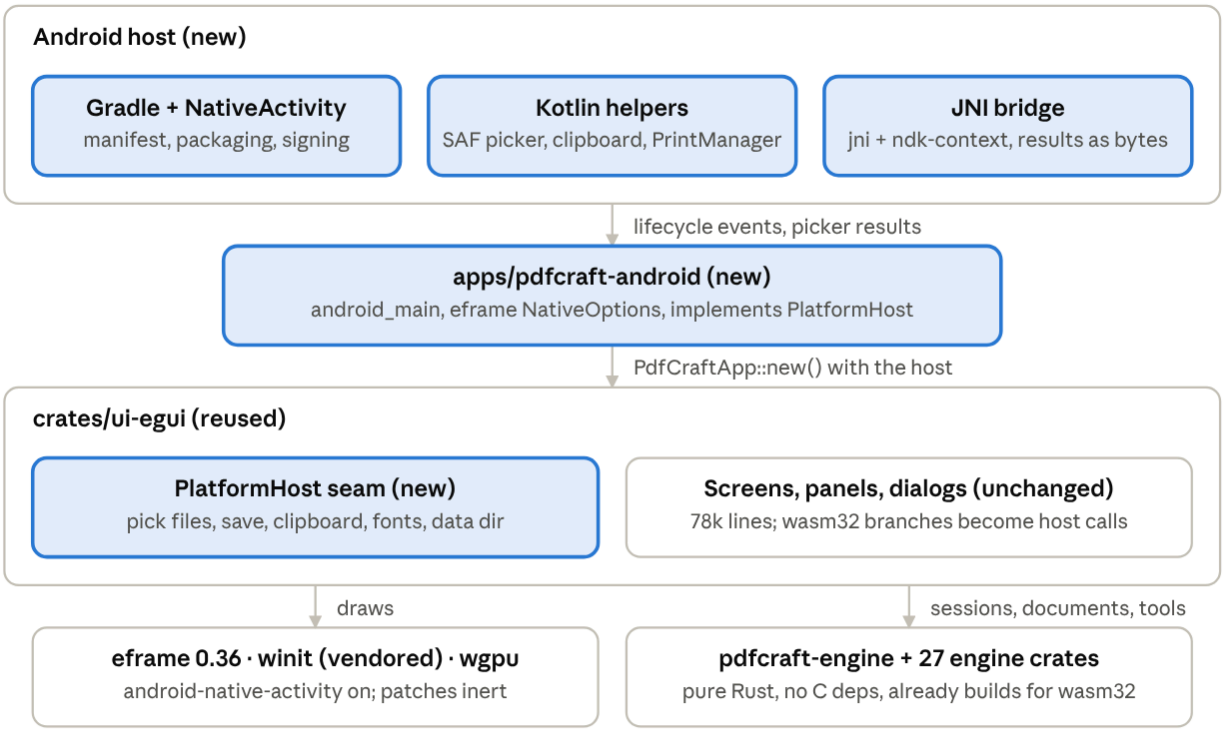
Parameter	Value	Reason
Platform	Android, `minSdk` 26 (Android 8.0), `targetSdk` 35	26 is the floor for Vulkan on wgpu and for stable `ACTION_OPEN_DOCUMENT_TREE`; 35 is the current Play requirement
ABIs	`arm64-v8a` (required), `x86_64` (emulator, CI)	32-bit ARM is excluded; a PDF renderer wants the address space
Rendering backend	wgpu on Vulkan; GLES 3 via wgpu-gl or `glow` as fallback	Mirrors the desktop `PDFCRAFT_RENDERER=gl` fallback
Form factors	Tablet (≥ 600 dp wide) first; phone layout in a later phase	The shell is desktop-shaped; see Work breakdown
Activity model	One `NativeActivity` hosting the egui surface, Rust owns the loop	What eframe's Android support expects
App identity	New name, package id and icon chosen by the owner	ArtCraft name and logos are trademarks and must be removed from a fork
Distribution	Sideload APK for development; AAB for Play	Owner decision on Play listing

The package id, display name and icon are reserved owner decisions (see Risks and decisions). Until they are set, the executor uses `app.pdfcraft.android` and a plain placeholder icon generated from a primitive shape, never the ArtCraft artwork.

Architecture of the port

The port adds one new app crate and one trait, and leaves the 137k-line engine and the 78k-line shell structurally untouched. The pattern to copy is `apps/pdfcraft-web/src/main.rs`: a thin entry point that constructs `PdfCraftApp::new()`, hands it an `eframe` runner, and wires a handful of host hooks (`startup_inbox`, `failed_inbox`, `host_save`, `host_dirty`).

Only the Android host and one seam are new; the shell and engine are reused



Tinted = new code. Plain = reused unchanged. Not built for Android: apps/pdfcraft, pdfcraft-cli, automation, print spooler.

port architecture · 2 new layers, 3 reused

The Android host owns the activity and anything that needs a Java API; the Rust crate owns everything else. All data crosses JNI as bytes and strings, never as file paths.

The `PlatformHost` seam. Today the shell branches on `cfg(target_arch = "wasm32")` at 270 sites, and native code reaches for `rfd`, `arboard` and `std::fs` directly. Adding a third `cfg(target_os = "android")` branch everywhere is the fast route but triples the maintenance surface. The specified route is a trait in `crates/ui-egui/src/platform_host.rs`:

```
pub trait PlatformHost: Send + Sync {
    fn pick_files(&self, req: PickRequest, reply: Box<dyn FnOnce(Vec<(String,
Vec<u8>)>) + Send>);
    fn save_file(&self, name: &str, mime: &str, bytes: Vec<u8>, reply:
Box<dyn FnOnce(SaveResult) + Send>);
    fn pick_folder(&self, reply: Box<dyn FnOnce(Option<FolderHandle>) +
Send>);
    fn clipboard_text(&self) -> Option<String>;
    fn set_clipboard_text(&self, text: &str);
    fn clipboard_image(&self) -> Option<RgbaImage>;
    fn data_dir(&self) -> Option<PathBuf>;
    fn fallback_font_paths(&self) -> Vec<PathBuf>;
    fn open_url(&self, url: &str);
    fn print(&self, pdf: Vec<u8>, job_name: &str) -> Result<(),
PrintUnsupported>;
}
```

Three implementations: `DesktopHost` (wraps the existing `rfd` / `arboard` / `directories` code, behaviour-identical to today), `WebHost` (wraps the existing `web-sys` code), and `AndroidHost` in the new app crate (JNI). The 45 `rfd` call sites and the clipboard, font and storage sites in `ui-egui` are rewritten to call `self.host`. This is the single largest task in the port and the one most worth upstreaming, since it also simplifies the web build.

Dependency gating. `ui-egui`'s `cfg(not(target_arch = "wasm32"))` dependency block becomes `cfg(not(any(target_arch = "wasm32", target_os = "android")))` for `rfd` and `arboard`; `pollster`, `skrifa` and `getrandom` stay for all native targets. The Android crate adds `jni = "0.21"`, `ndk-context = "0.1"`, `android-activity` (via eframe's feature), `android_logger`. The workspace `default-members` list is left as is; Android is built with an explicit `-p pdfcraft-android`.

What is reused unchanged: every engine crate, `pdfcraft-render` including its thread pool, the vendored `hayro` / `lopdf` patches, all screens and panels in `ui-egui`, settings persistence through `eframe::Storage` (pointed at the app's internal files dir).

What is replaced: `apps/pdfcraft` (desktop entry, single-instance, update check, GPU selection), `rfd` dialogs, `arboard`, the system-font path list, the print spooler, `pdfcraft-platform`'s desktop theme reader (Android exposes dark mode through the activity's configuration, read once at start and on `onConfigurationChanged`).

Work breakdown

Seven phases, strictly ordered; each is a separate branch and pull request, merged only when its acceptance criteria pass. Phase 0 is a day; Phases 1–3 are the core (roughly three to four weeks of agent time with owner review); Phases 4–6 take the app from "works" to "shippable".

Phase 0 — Prove the engine compiles for Android

Nothing new is written. The goal is to learn which crates fail and why before any design is committed to.

- [ ] Install Rust target `aarch64-linux-android` and `x86_64-linux-android`; install `cargo-ndk`; install NDK r27 or newer.
- [ ] Run `cargo ndk -t arm64-v8a build -p pdfcraft-engine --release` and record the result.
- [ ] Run the same for `pdfcraft-render`, `pdfcraft-ocr`, `pdfcraft-sign`, `pdfcraft-js`, `pdfcraft-print`, `pdfcraft-platform`.
- [ ] Run `cargo ndk -t arm64-v8a build -p pdfcraft-ui-egui` and record every error; this is expected to fail on `rfd`, `arboard` and possibly `eframe` features.
- [ ] Write the findings to `docs/android/phase0-report.md`: one table, crate × result × error class.

Acceptance: every engine crate builds for `aarch64-linux-android` with no source change, or each failure has a one-line diagnosis and a fix plan in the report. If any engine crate needs more than a feature flag to build, stop and report to the owner before Phase 1.

Phase 1 — A blank egui window on a device

Create `apps/pdfcraft-android` and the Android project around it, and get eframe drawing a trivial UI on a real device.

- [ ] `apps/pdfcraft-android/Cargo.toml`: `crate-type = ["cdylib"]`, depends on `eframe` with `android-native-activity` (through `winit`), `wgpu` and `glow`; `android_logger`, `log`, `ndk-context`, `jni`. Not yet on `pdfcraft-ui-egui`.
- [ ] `src/lib.rs`: `#[no_mangle] fn android_main(app: AndroidApp)` that sets `NativeOptions { android_app: Some(app), .. }` and runs an `eframe::App` showing one label and a button.
- [ ] `android/` directory at the repo root: Gradle project (`settings.gradle.kts`, `app/build.gradle.kts`), `AndroidManifest.xml` declaring one `NativeActivity` with `android.app.lib_name = pdfcraft_android`, `hasCode = true` (a Kotlin helper class is coming in Phase 2), `minSdk 26`, `targetSdk 35`.

- [ ] `xtask android` subcommand (the repo already has `xtask`) that runs `cargo ndk -t arm64-v8a -t x86_64 -o android/app/src/main/jniLibs build -p pdfcraft-android --release` and then `./gradlew assembleDebug`.
- [ ] Verify on a device and on an `x86_64` emulator: the window appears, the button responds to touch, rotation does not crash, backgrounding and resuming does not crash.
- [ ] If wgpu fails to create a Vulkan surface on the test device, try `Renderer::Glow`; record which backend each test device uses in `docs/android/devices.md`.

Acceptance: `cargo xtask android` produces an installable debug APK from a clean clone; the placeholder app runs on at least one physical arm64 device and the emulator without a crash through launch, touch, rotate, background, resume.

Phase 2 — The `PlatformHost` seam and the Android host

The refactor that lets `ui-egui` build for Android. Done in two pull requests: 2a refactors the shell with no behaviour change on desktop and web; 2b adds the Android implementation.

2a (shell refactor, desktop and web stay green):

- [ ] Add `crates/ui-egui/src/platform_host.rs` with the `PlatformHost` trait from the Architecture section, plus `PickRequest { purpose: FilePurpose, multiple: bool, extensions: Vec<&'static str>, title: String }`, `SaveResult`, `FolderHandle`.
- [ ] Add `host: Arc<dyn PlatformHost>` to `PdfCraftApp`; `PdfCraftApp::new()` becomes `PdfCraftApp::new(host)`. Desktop and web `main.rs` pass `DesktopHost` / `WebHost`.
- [ ] `DesktopHost` in `crates/ui-egui/src/hosts/desktop.rs`: move the existing `rfd::AsyncFileDialog` construction (`files.rs::files_picker` and the per-feature dialogs), `arboard` calls, `eframe::storage_dir`, and the `system_fonts.rs` path tables behind it.
- [ ] `WebHost` in `crates/ui-egui/src/hosts/web.rs`: move the existing `web-sys` download and `rfd` wasm picker code behind it.
- [ ] Rewrite the 45 `rfd` call sites in `files.rs`, `combine_ui.rs`, `create_ui.rs`, `ocr_ui.rs`, `print_ui.rs`, `actions_ui.rs`, `content_ui.rs`, `marks_ui.rs`, `measure_ui.rs`, `comment_props.rs` to call `self.host.pick_files(...)` / `save_file(...)` / `pick_folder(...)`. The existing `pickers::Ask` enum and the `FileRequest` / `use_files` flow are kept as the delivery mechanism; only the dialog construction moves.
- [ ] Rewrite the clipboard sites (`Create ▸ Clipboard`, copy text) to call the host.
- [ ] Move `rfd` and `arboard` under `cfg(not(any(target_arch = "wasm32", target_os = "android")))` in `ui-egui/Cargo.toml`.

- [ ] Run the full desktop test suite (`cargo test` in the workspace, including the `egui_kittest` UI tests) and the web build (`trunk build --release` in `apps/pdfcraft-web`). Both must pass unchanged.

2b (Android host):

- [ ] `android/app/src/main/java/.../PdfCraftActivity.kt` extending `NativeActivity`: `openDocuments(mimeTypes, multiple)`, `createDocument(name, mime)`, `openDocumentTree()`, each firing an Intent and, in `onActivityResult`, reading the content URI through `ContentResolver` into a `ByteArray` and calling a `native` method with `(requestId, name, bytes)`. Also `getClipboardText()`, `setClipboardText()`, `getFilesDir()`, `isDarkMode()`, `openUrl()`.
- [ ] `apps/pdfcraft-android/src/host.rs`: `AndroidHost` implementing `PlatformHost` over `jni`; pending callbacks keyed by `requestId` in a `Mutex<HashMap<u64, Box<dyn FnOnce..>>>`; the JNI `Java_..._onFilesPicked` entry point resolves them and calls `ctx.request_repaint()`.
- [ ] `data_dir()` returns the activity's `filesDir`; eframe persistence is pointed there through `NativeOptions::persistence_path`.
- [ ] `fallback_font_paths()` returns `/system/fonts/NotoSans*.ttf`, `NotoSansCJK*`, `NotoSansDevanagari*` and so on; verify the paths exist on API 26, 30 and 35 emulators.
- [ ] Handle an incoming `ACTION_VIEW` Intent (user taps a PDF in Files or Gmail): read the URI in `onCreate` / `onNewIntent` and push it to `startup_inbox`, as the web shell does with `?file=`.
- [ ] `apps/pdfcraft-android` now depends on `pdfcraft-ui-egui`; `android_main` constructs `PdfCraftApp::new(Arc::new(AndroidHost::new(app)))`.

Acceptance: Open a PDF through the picker, scroll it, tap Organize, rotate a page, Save As to a new file through `createDocument`, reopen that file in another app and see the rotation. Desktop and web test suites still pass. No `rfd` or `arboard` symbol in the Android library (`nm -D libpdfcraft_android.so | grep -i rfd` is empty).

Phase 3 — Gate out desktop-only features

Everything that cannot work on Android degrades to a visible, non-crashing state.

- [ ] Print: `print()` on `AndroidHost` either hands the print-ready PDF to Android's `PrintManager` (`PrintDocumentAdapter` over a `PdfRenderer`) or returns `PrintUnsupported`; the Print dialog shows "Printing is not available on this device" in the latter case. Printer enumeration in `print_ui.rs` is skipped on Android.
- [ ] Automation / `--control` channel: compile-time excluded (`cfg(not(target_os = "android"))` on the module).

- [ ] Help ▸ Check for updates, Discord button, portable mode, single instance: not present in the Android shell (they live in `apps/pdfcraft` and are simply not ported).
- [ ] `pdfcraft-platform`: `desktop_theme` returns the activity's dark-mode flag on Android; `staging` keeps its temp-dir logic but under `cacheDir`.
- [ ] Audit every remaining `std::fs::` and `std::env::` call in `ui-egui` (182 sites with `thread::spawn`): each must either go through `host.data_dir()` / `cacheDir`, or be behind a `cfg` that excludes Android. Document each decision in `docs/android/fs-audit.md`.
- [ ] Crash recovery (unsaved-work autosave) writes under `data_dir()/recovery` and is verified to restore after the OS kills the process in the background.

Acceptance: Every menu item and tool in the catalogue either works or shows a clear "not available on Android" state; a scripted walk through the tool catalogue (`pdfcraft-engine`'s command list) on the emulator produces no panic. Process-death recovery restores an edited, unsaved document.

Phase 4 — Touch and tablet layout

- [ ] Pinch-to-zoom and two-finger pan on the page view (egui provides `multi_touch()`; wire it to the existing zoom and scroll state).
- [ ] Minimum hit target 44 dp for toolbar buttons, tab close buttons and panel resize handles; egui style `spacing.button_padding` and `interact_size` set per `ctx.pixels_per_point()` on Android.
- [ ] `pixels_per_point` derived from `DisplayMetrics.density` through the activity, not winit's default.
- [ ] On-screen keyboard: show/hide through `AndroidApp::show_soft_input` when a text field gains or loses focus; the page view scrolls so the focused form field stays visible.
- [ ] Android Back button: closes the topmost dialog or panel; at the root, prompts if there is unsaved work.
- [ ] Tablet layout (≥ 600 dp): the existing three-column shell is kept; the left tools panel collapses by default below 840 dp.
- [ ] Long-press opens the context menu that right-click opens on desktop.

Acceptance: A reviewer on a 10-inch tablet can open, read, search, comment, organize and save without a keyboard or mouse. Every interactive element passes an automated hit-size check (an `egui_kittest` test that walks the widget tree).

Phase 5 — Phone layout

- [ ] Below 600 dp: single-column mode. Left and right panels become bottom sheets or full-screen overlays; the top toolbar becomes a bottom action bar with overflow.

- [ ] Organize grid: two columns on phones, thumbnails sized to width.
- [ ] Comment panel as a full-screen overlay with its own Back handling.
- [ ] Landscape on phone: two-pane if ⩾ 600 dp wide, else single-column.

Acceptance: The Phase 4 reviewer task passes on a 6-inch phone. Owner decides whether phone support ships in v1 or is deferred (see Decisions).

Phase 6 — Ship

- [ ] Remove all ArtCraft brand assets from the Android build and set the owner's name, package id and icon.
- [ ] Release signing config, `proguard` /R8 rules (the Kotlin side is tiny; mostly "keep native methods"), AAB build.
- [ ] CI: a GitHub Actions job `android.yml` that installs NDK and SDK, runs `cargo xtask android --release`, runs the `x86_64` emulator smoke test, and uploads the APK as an artifact.
- [ ] Performance budget check (see Testing).
- [ ] Play Store listing assets, privacy declaration ("no account, no telemetry" carries over), data-safety form.

Acceptance: CI is green on `main`; the AAB installs from Play internal testing; the Definition of done in section 1 holds.

Build and toolchain

One command, `cargo xtask android`, must take a clean clone to an installable APK; everything below is what that command needs.

Component	Version / setting	Notes
Rust	stable ⩾ 1.95 (workspace `rust-version`); `rustup target add aarch64-linux-android x86_64-linux-android`	The workspace is edition 2024
`cargo-ndk`	⩾ 3.5	Sets the linker and `CC` / `AR` for each ABI; `cargo ndk -t arm64-v8a -t x86_64 -o android/app/src/main/jniLibs build`

Component	Version / setting	Notes
Android NDK	r27 (LTS)	Set `ANDROID_NDK_HOME`; r27's clang handles the `aws-lc-sys`-free tree without extra flags
Android SDK	platform 35, build-tools 35.0.0, cmdline-tools	`ANDROID_HOME`
Gradle	8.x with the Android Gradle Plugin 8.x, Kotlin 2.x	`android/` is a standard project; the only Kotlin is the activity helper
JDK	17	AGP 8 requirement
Rust library	`cdylib`, `libpdfcraft_android.so`, `panic = "abort"` kept off so panics are logged	`android_logger` routes `log` to logcat with tag `pdfcraft`
Profiles	`release` with the workspace's `lto = "thin"`, `codegen-units = 1`; `opt-level = "s"` is NOT used (render speed matters more than size)	Expect ~25–35 MB per ABI before stripping; `strip = true` in release
Vendored crates	`vendor/winit`, `vendor/egui-wgpu` stay patched; enable `android-native-activity` on the vendored winit through eframe's feature	Do not fork or modify the vendored copies for Android

Cargo configuration. `.cargo/config.toml` already exists in the repo; add per-target linker settings only if `cargo-ndk` does not provide them. No `build.rs` changes are expected; `apps/pdfcraft/build.rs` (Windows resources) is not part of the Android crate.

Feature flags on eframe for Android: `default_fonts`, `wgpu`, `glow`, `persistence`, `android-native-activity`. Not `accesskit` initially (AccessKit's Android adapter is experimental; revisit in Phase 4 if it has stabilised), not `wayland` / `x11`.

CI (`.github/workflows/android.yml`): Ubuntu runner; cache `~/.cargo` and `android/.gradle`; install NDK via `sdkmanager`; run `cargo xtask android --release`; start an `x86_64` API 30 emulator (`reactivecircus/android-emulator-runner` or equivalent) and run the smoke test from the Testing section; upload the APK. Expected wall time: 25–40 minutes cold, under 15 warm. The existing desktop CI jobs are left unchanged and must stay green after Phase 2a.

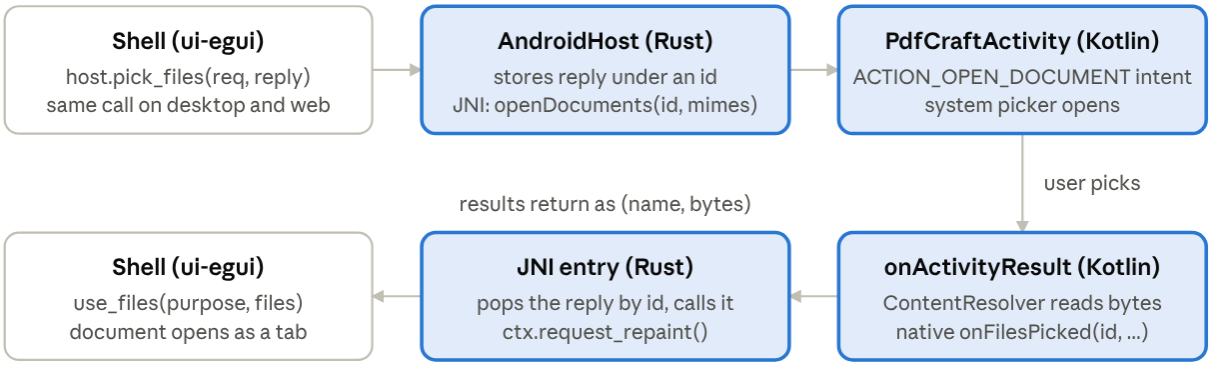
Signing: a debug keystore for development; release signing through a keystore the owner holds, injected via CI secrets (`ANDROID_KEYSTORE_B64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`). The executor never generates or commits a release key.

Platform integration details

Every Android-specific behaviour lives in two places: `PdfCraftActivity.kt` for anything that needs a Java API, and `apps/pdfcraft-android/src/host.rs` for the Rust side of the bridge. The shell calls the `PlatformHost` trait and knows nothing else.

Files (Storage Access Framework). Android apps do not get filesystem paths to user documents; they get content URIs from the system picker. The round trip for opening is below; saving is the mirror image.

Files cross JNI as bytes; the shell never sees an Android path



Tinted = Android-only code. Save follows the same path with ACTION_CREATE_DOCUMENT and the bytes going the other way.

file picker round trip · 6 steps across JNI

Rules the executor must follow:

- Read the whole document into memory on the Kotlin side with `contentResolver.openInputStream(uri).readBytes()`. PdfCraft's `cos` layer parses from a byte slice, so this matches how the web build already works. Files over 200 MB get a confirmation dialog first.
- Keep the content URI alongside the bytes (`FileRequest` gains an opaque `origin: Option<String>`), and take a persistable permission (`takePersistableUriPermission`) so Save (not Save As) can write back to the same document with `ACTION_OPEN_DOCUMENT`'s `FLAG_GRANT_WRITE_URI_PERMISSION`. The existing "validate PDF output before replacing files" step (commit `c1a41cc`) is kept: write to a temp file in `cacheDir`, validate, then copy over the URI's output stream.

- Save As uses `ACTION_CREATE_DOCUMENT` with the proposed name and `application/pdf`; the export dialogs pass `text/csv`, `application/xml` and so on.
- Folder exports (split, extract pages to a folder) use `ACTION_OPEN_DOCUMENT_TREE` and `DocumentsContract.createDocument` per file; `FolderHandle` wraps the tree URI.
- `ACTION_VIEW` and `ACTION_SEND` intents (PDF opened from Files, Gmail, a browser) are declared in the manifest for `application/pdf`; the URI is read in `onCreate` / `onNewIntent` and pushed to `startup_inbox`.
- Recent files: store the display name and the persisted URI, not a path; the recent-files list already tolerates missing entries.

Storage directory. `data_dir()` returns `context.filesDir` (`/data/data/<pkg>/files`). eframe persistence (`app.ron`), logs and crash recovery go there. Temporary staging goes to `context.cacheDir`, which the OS may clear.

Clipboard. `ClipboardManager` through JNI for text. Image paste (Create ▸ Clipboard) reads a `content://` image URI from the primary clip if present; otherwise the feature reports "no image on the clipboard". Copying rendered images out is deferred to a later version.

Fonts. egui's bundled fonts cover Latin; script fallback (CJK, Devanagari, Arabic, Hebrew, Thai) comes from `/system/fonts/`. The host returns the Noto files present on the device; `system_fonts.rs` already validates each candidate with `skrifa` before use, so missing files are harmless. Emoji use `NotoColorEmoji.ttf` if egui's version supports colour bitmaps, otherwise fall back to monochrome.

Theme. `isDarkMode()` reads `Configuration.uiMode & UI_MODE_NIGHT_MASK`; the activity calls a native `onThemeChanged(dark)` from `onConfigurationChanged` so the app follows the system without restart.

Lifecycle. `android-activity` delivers `Resumed` / `Paused` / `Destroyed` through winit's `ApplicationHandler` on the vendored winit; eframe handles surface loss on `Paused`. The app must: save settings on `Paused` (not only on close); flush crash recovery on `Paused`; and treat `onTrimMemory` as a cue to drop rendered-page caches (expose `PdfCraftApp::trim_caches()`; the raster cache in `pdfcraft-render` already has an eviction path).

Back button and gestures. Handle `KeyEvent.KEYCODE_BACK` (and the predictive-back callback on API 33+) by calling into Rust `on_back_pressed() -> bool`; the shell closes the topmost modal, panel or tab, and returns `false` at the root so the activity finishes. Gesture navigation is left to the system; the app does not request exclusion zones.

Soft keyboard. `AndroidApp::show_soft_input(true/false)` when egui reports a text field gaining or losing focus (`ctx.memory().focused()` changes); the page view adds bottom padding equal to the IME inset, read from `WindowInsets` through the activity.

Printing. `PrintManager.print(jobName, PrintDocumentAdapter, attributes)` where the adapter's `onWrite` copies the print-ready PDF that `pdfcraft-print` already produces for desktop. This replaces the CUPS and PowerShell spoolers with roughly 80 lines of Kotlin and is the one desktop feature that gets a real Android implementation rather than a stub.

Permissions. None at install time. No `READ_EXTERNAL_STORAGE`; SAF grants access per document. `INTERNET` is not requested: the desktop update check is not ported, and the app makes no network calls.

Out of scope for v1

These are deliberately not built. Each has a reason and a visible state in the app so a user is never left guessing.

Feature	v1 state	Reason
Automation / MCP server / `--control` channel	Not compiled in	Desktop agent tooling; no use on a phone
Help ▸ Check for updates	Absent	Play Store handles updates; keeps the app offline
Single-instance handoff, `--new-window`	Absent	Android has one task per app by default
Portable mode (`PdfCraftData` beside the executable)	Absent	No concept on Android
Desktop GPU selection (`compositor_gpu.rs`, Windows `EnumDisplayDevices`)	Absent	One GPU on a phone
Copying rendered images to the clipboard	"Not available" message	Needs a `ContentProvider`; deferred
Drag-and-drop of files onto the window	Absent	Rare on tablets; the picker covers it
Keyboard shortcuts dialog	Hidden unless a hardware keyboard is attached	Shortcuts still work with a keyboard

Feature	v1 state	Reason
Phone (< 600 dp) layout	Phase 5, owner's call	Tablet-first scope
Compare, Preflight (PDF/A), Export to Word on very large files	Work, with a size warning above 50 MB	Memory headroom on phones is lower; measure in Phase 6
OCR on devices with < 4 GB RAM	Works but warns about speed	`rten` model inference is CPU-bound; measured in Testing
AccessKit (screen reader support)	Deferred	AccessKit's Android adapter was not mature as of the eframe 0.36 release; re-evaluate at Phase 4

Testing and verification

The existing desktop and web test suites are the regression net for the shell refactor; the Android build adds a smoke test that runs on the emulator in CI and a manual checklist on real devices.

Regression (every phase from 2a on): `cargo test --workspace` on Linux, including the `egui_kittest` UI tests in `crates/ui-egui`, and `trunk build --release` in `apps/pdfcraft-web`. A phase is not merged while either is red.

Android smoke test (CI, `x86_64` emulator, API 30): an instrumented run that launches the activity with an `ACTION_VIEW` intent on `parity/` test PDFs (the repo keeps a corpus there), waits for the first rendered frame, takes a screenshot, and checks logcat for `panicked at`. From Phase 2b it also drives the picker through `UiAutomator` to open and save one document. From Phase 3 it walks the tool catalogue through the existing control-channel command list, with the channel compiled in only for the test build.

Device matrix (manual, each phase from 1):

Device class	Example	Backend to verify	Why
Flagship tablet	Pixel Tablet or Galaxy Tab S9	Vulkan	The primary target
Mid-range phone, 2021	Pixel 5a / Galaxy A52	Vulkan, then GLES fallback forced	Most common hardware
Low-end, API 26–28	any 3 GB device	GLES	Floor of the support range

Device class	Example	Backend to verify	Why
Emulator `x86_64` API 30 and 35	Android Studio AVD	SwiftShader GLES	CI parity

Performance budgets (measured on the mid-range phone, release build, recorded in `docs/android/perf.md`):

Measure	Budget	How measured
Cold start to first frame	≤ 1.5 s	`adb shell am start -W`
Open and first page of a 10 MB text PDF	≤ 1.0 s	logcat timestamps from the engine's existing `log` lines
Scroll at fit-width, 13-page showcase PDF	≥ 55 fps sustained, no frame > 50 ms	`adb shell dumpsys gfxinfo`
Peak RSS with a 50 MB scanned PDF open	≤ 600 MB	`dumpsys meminfo`
OCR of one 300-dpi page	≤ 8 s	app timer; informs the OCR warning threshold
APK size per ABI (stripped)	≤ 40 MB	`ls -l`

A budget miss is reported, not silently accepted: the executor records the number, the likely cause and a proposed fix, and the owner decides whether it blocks the phase.

Process death: `adb shell am kill <pkg>` with an unsaved edit, relaunch, confirm the recovery prompt restores it (Phase 3 acceptance).

Static checks: `cargo clippy --target aarch64-linux-android -p pdfcraft-android -- -D warnings` (the workspace lints apply); `cargo deny check` still passes with the new dependencies (`jni`, `ndk-context`, `android_logger` are MIT/Apache); no new `unsafe` anywhere (`unsafe_code = "forbid"` is workspace-wide; JNI's `unsafe` surface is confined to the `jni` crate's own API, which exposes safe wrappers for everything this port needs).

Risks, unknowns and owner decisions

The port has no known blocker; the risks below are the ones that could change its shape or cost, ordered by how early they are found out.

Risk	Found in	Impact if it lands	Mitigation
An engine crate fails to build for Android (e.g. `getrandom` backend, `boa` or `rten` using a platform API)	Phase 0, day 1	Days, not weeks: feature flags or a vendored patch	Phase 0 exists to find this before anything is designed around it
wgpu cannot create a Vulkan surface on a test device; `glow` fallback is slower	Phase 1	Scroll fps budget missed on low-end devices	Backend chosen at start-up per device, same pattern as desktop's `PDFCRAFT_RENDERER=gl`
The `PlatformHost` refactor regresses desktop or web behaviour	Phase 2a	Upstream drift, user-visible bugs	2a is a separate PR with no behaviour change; the desktop and web test suites gate it
The vendored `winit` / `egui-wgpu` patches conflict with the Android feature or a future rebase	Phase 1, then ongoing	Build breaks when the fork rebases on upstream	Enable the feature through eframe only; never edit the vendored copies for Android
Memory: a 50 MB scanned PDF read whole into a byte buffer plus raster caches exceeds the phone's budget	Phase 3 and 6	OOM kills on 3–4 GB devices	`trim_caches()` on `onTrimMemory`; size warning above 50 MB; measure, do not guess
OCR is too slow on phones to be useful	Phase 6	Feature ships with a warning or is tablet-only	Measured against the 8 s budget; `rten` supports quantised models if needed
The egui shell on a phone is usable but feels like a shrunken desktop app	Phase 5	Store reviews, adoption	Tablet-first scope; phone layout is a separate, optional phase
Upstream `storytold/pdfcraft` moves fast (daily commits as of 2026-10-14) and the fork drifts	Ongoing	Merge pain	Keep Android changes additive and in new files where possible; upstream the `PlatformHost` seam

Decisions reserved for the owner. The executor stops and asks at these points rather than choosing:

1. App name, package id and icon (needed before Phase 1 is merged; a placeholder is used until then). The ArtCraft brand may not be used.
2. Whether phone layouts (Phase 5) are in v1 or deferred.
3. Whether printing goes through `PrintManager` in v1 or is stubbed (Phase 3).
4. Whether to pursue upstreaming the `PlatformHost` refactor to `storytold/pdfcraft` now (opens a PR against upstream after 2a) or later.
5. Minimum SDK: 26 as specified, or 24 if a target market needs it (costs the Vulkan floor; GLES only below 26).
6. Play Store listing: publish, or sideload only.
7. Any performance budget miss (see Testing) is a decision, not an automatic failure.

Working rules for the executing agent

The repository's own `CLAUDE.md` and `AGENTS.md` apply in full; the points below are the ones this port touches, plus two places where the port needs an explicit exception.

From the repo's rules (read both files before the first commit):

- Never panic. Every picker result, intent, JNI string and settings file is untrusted input. `Result` and `?`, `get()` not `[i]`, checked arithmetic, capped allocations (a 2 GB file from the picker must be refused, not read). No `unwrap` / `expect` outside tests; new crates start with `#![deny(clippy::unwrap_used, clippy::expect_used, clippy::panic, clippy::unimplemented, clippy::todo, clippy::unreachable)]`.
- No `unsafe` (`unsafe_code = "forbid"`). Use the `jni` crate's safe API; if a JNI call needs `unsafe`, stop and ask the owner. Never relax the workspace lint.
- Layering: nothing below `ui-egui` may depend on egui, winit, eframe or `rfd`. The `PlatformHost` trait lives in `ui-egui`; the engine crates stay untouched. `cargo xtask layers` must pass.
- Assets: no system fonts are embedded, committed or shipped. Reading `/system/fonts/` at runtime as the last interface fallback is the same allowance `system_fonts.rs` already uses on desktop (`AGENTS.md` §1.4) and nothing from it may enter a PDF or the repository. The app icon is contributor-original and gets an `ATTRIBUTION.toml` entry; `cargo xtask assets` must pass.
- The ArtCraft name and logos are removed from the Android build and manifest (`AGENTS.md` §1.2).
- Quality gates before every commit, scoped to the crates touched: `cargo fmt --check`, `cargo clippy -p <crate> --all-targets -- -D warnings`, `cargo test -p <crate>`, `cargo xtask layers`, `cargo xtask assets`. The full workspace sweep is CI's job.

- Prove a test fails before the fix by reverting the fix.
- Rebase as a review step: `main` moves daily; read every resolved hunk.
- Record progress: add a line to the `ROADMAP.md` progress log per merged phase; keep `parity/` honest (an Android build does not change a feature's parity status, but a feature stubbed on Android gets a note).

Exceptions this port needs (owner approves before Phase 1):

1. `CLAUDE.md` says "Rust only in the product and build, no handwritten JS/TS". The Android activity helper is Kotlin, about 150 lines, confined to `android/app/src/main/java/`. It exists because SAF, `PrintManager`, `ClipboardManager` and the IME insets are Java APIs with no safe pure-Rust binding. Everything it does is a thin forwarder; no logic lives there. This is the same shape as the `index.html` and trunk loader the web build already tolerates.
2. "Everything is reachable headlessly" (`AGENTS.md` §3): the `automation` crate is not compiled into the Android build. Features are still headless through the desktop CLI; the Android app adds no features of its own, so no new tools are owed.

Branch and commit hygiene:

- One branch per phase, named `android/phase-N-<slug>`; Phase 2 is two branches (`android/phase-2a-platform-host`, `android/phase-2b-android-host`).
- Commits follow the repo's conventional style seen in `git log` (`fix(save): ...`, `feat(ui): ...`); use the `android` scope for the new crate and `ui` for shell changes.
- Every pull request description states which acceptance criteria from the Work breakdown were verified, on which device, with the command or screenshot that proves it.
- Android-only files go under `apps/pdfcraft-android/`, `android/` and `docs/android/`; shared files are changed additively. A diff to a shared file that is not explained by the `PlatformHost` refactor or a `cfg` gate needs a sentence in the PR saying why.
- Never commit a release keystore, a `local.properties`, or the `android/.gradle` and `android/app/build` directories (add them to `.gitignore` in Phase 1).
- When blocked by a reserved decision, open the question in the PR, mark the task as blocked, and move to the next unblocked task in the same phase; do not guess.